



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
GENETIC ALGORITHMS



PRÁCTICA 8
MEXICAN WAVE

NOMBRE DEL ALUMNO: GARCÍA QUIROZ GUSTAVO IVAN

GRUPO: 6CV2

NOMBRE DEL PROFESOR: ROSAS TRIGUEROS JORGE LUIS

FECHA DE REALIZACIÓN DE LA PRÁCTICA: 05/09/2024

FECHA DE ENTREGA DEL REPORTE: 12/10/2024

Índice

1	Marco teórico.	1
1.1	Ola Mexicana	1
2	Material y equipo.	3
2.1.1	Hardware	3
2.1.2	Software.....	3
3	Desarrollo de la práctica.	4
3.1	Actividades desarrolladas	4
3.1.1	Diseño e Implementación del Código	4
3.1.2	Inicialización y Matrices de Influencia	4
3.2	Proceso de Simulación	5
3.2.1	Calcular influencia entre espectadores	5
3.2.2	Actualización los estados del sistema.....	6
3.2.3	Visualización de ola mexicana	7
3.3	Desafíos y soluciones	7
3.4	Resultados obtenidos.....	8
4	Conclusiones y recomendaciones.	13
5	Bibliografía	14

1 Marco teórico.

1.1 Ola Mexicana

El fenómeno de la "Ola Mexicana" o "La Ola" surgió a la fama durante la Copa Mundial de 1986 celebrada en México. Esta consiste en una acción coordinada entre los espectadores de un estadio, donde en una sección se ponen de pie con los brazos levantados, y luego se sientan mientras la siguiente sección se levanta para repetir el movimiento, creando así una "ola" que se propaga a través de las gradas.



Figura 1 "Ola Mexicana".

Para entender y cuantificar este comportamiento colectivo humano, los autores utilizaron una variante de modelos desarrollados originalmente para describir medios excitables, como el tejido cardíaco. Al modelar la reacción de la multitud a los intentos de desencadenar la ola, se reveló cómo se estimula este fenómeno y puede resultar útil para controlar eventos que involucren grupos de personas entusiasmadas.

Mediante el análisis de grabaciones de video de 14 olas en estadios de fútbol con más de 50,000 espectadores, los autores encontraron que la ola suele moverse en dirección de las agujas del reloj a una velocidad de aproximadamente 12 metros (o 20 asientos) por segundo, y tiene un ancho de entre 6 y 12 metros (correspondiente a un ancho promedio de 15 asientos). La ola es generada por no más de unas pocas decenas de personas que se ponen de pie simultáneamente, y luego se expande a través de toda la multitud adquiriendo una forma lineal casi estable.

Los autores desarrollaron dos modelos de simulación matemática, uno minimal y otro más detallado, para reproducir y predecir los detalles de este comportamiento colectivo. Estos modelos se basan en analogías con los modelos de medios excitables, donde se considera a las personas como unidades excitables que

pueden ser activadas por un estímulo exterior (una concentración ponderada por la distancia y la dirección de personas activas cercanas que excede un valor umbral). Una vez activadas, cada unidad sigue un conjunto de reglas internas para pasar por las fases activa (de pie y agitando) y refractaria (pasiva) antes de volver a su estado de reposo original (excitable).

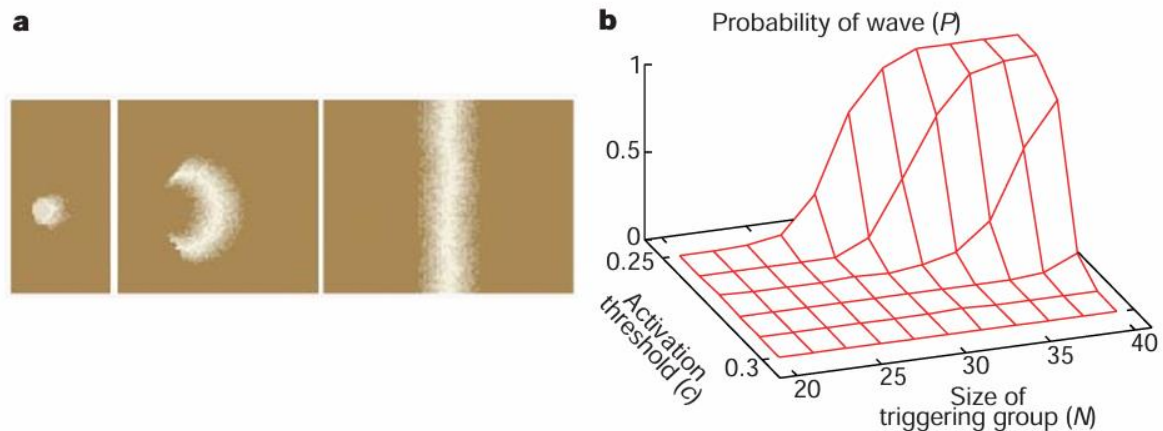


Figura 2 Simulación matemática de la "Ola Mexicana".

Los resultados de estos modelos indican que la dependencia de la eventual ocurrencia de una ola con respecto al número de iniciadores es una función que cambia abruptamente, es decir, desencadenar una ola mexicana requiere una masa crítica de iniciadores. Este enfoque podría tener implicaciones para el control de multitudes, por ejemplo, en incidentes violentos asociados con manifestaciones o eventos deportivos, donde es esencial entender las condiciones bajo las cuales pequeños grupos pueden ganar el control de la multitud y cómo esta perturbación o transición en el comportamiento podría propagarse rápida y efectivamente.

2 Material y equipo.

Para esta práctica se usaron las siguientes herramientas de software y hardware necesarias para realizar la práctica.

2.1.1 Hardware

- Computadora.

2.1.2 Software

- Google colaboratory.
- Librería NumPy.
- Python 3.

3 Desarrollo de la práctica.

En esta práctica, se desarrolló un programa en Python para simular una ola mexicana con al menos 80 filas de individuos. El proyecto se centró en la implementación de una simulación visual que representara el fenómeno colectivo conocido como "ola", comúnmente visto en eventos deportivos.

3.1 Actividades desarrolladas

3.1.1 Diseño e Implementación del Código

La implementación de la simulación de la Ola Mexicana se desarrolló utilizando Python, empleando un enfoque orientado a objetos que se divide en dos clases principales: MexicanWave y WaveSimulationGUI. Este diseño permite una clara separación entre la lógica de simulación y la interfaz de usuario.

3.1.2 Inicialización y Matrices de Influencia

La clase MexicanWave constituye el núcleo de la simulación, encargándose de toda la lógica matemática y el procesamiento de estados. Esta clase implementa el modelo de medio excitable donde cada espectador puede encontrarse en diferentes estados (reposo, activo o refractario). Los parámetros clave se establecen en el constructor de la clase, permitiendo una fácil configuración y experimentación.

- Parámetros fundamentales:
 - rows y cols: Dimensiones del estadio (80 x 100)
 - n_a y n_r: Estados activos y refractarios (5 cada uno)
 - c y dc: Umbral de excitación y su variación (0.25 ± 0.05)
 - R: Radio de influencia entre espectadores (3)
 - w0: Peso direccional para la propagación (0.5)
- Estructuras de datos principales:
 - states: Matriz NumPy para el estado actual de cada espectador
 - thresholds: Matriz de umbrales individuales
 - distances: Matriz de distancias entre espectadores
 - directions: Matriz de direcciones para la propagación
- Métodos esenciales:
 - calculate_influence(): Calcula la influencia entre espectadores
 - update(): Actualiza los estados del sistema
 - trigger_wave(): Inicia la ola en una posición específica

La interfaz gráfica, implementada en la clase WaveSimulationGUI, proporciona una visualización interactiva de la simulación. Utiliza matplotlib para la representación gráfica y ipywidgets para los controles interactivos, permitiendo una observación clara del fenómeno y la interacción con la simulación.

El sistema de visualización emplea un mapa de calor (heatmap) que representa los diferentes estados de los espectadores mediante una escala de colores, donde las tonalidades más intensas indican estados activos y las más claras estados de reposo o refractarios. Esta representación permite observar claramente la propagación de la ola a través del estadio.

La implementación hace un uso extensivo de las capacidades de NumPy para operaciones matriciales eficientes, lo que permite una simulación fluida incluso con un gran número de espectadores. El código está estructurado de manera modular, facilitando futuras modificaciones o extensiones del modelo.

3.2 Proceso de Simulación

3.2.1 Calcular influencia entre espectadores

El proceso de simulación de la Ola Mexicana se ejecuta a través de una serie de pasos iterativos que modelan el comportamiento colectivo de los espectadores en el estadio. El principal función de este proceso reside en el método update() de la clase MexicanWave, que gestiona la evolución temporal del sistema.

La simulación comienza con el cálculo de la influencia entre espectadores, implementado en el método calculate_influence(). Este proceso es importante para determinar cómo cada persona afecta a sus vecinos y cómo se propaga la ola. El código utiliza matrices de NumPy para realizar estos cálculos de manera eficiente:

```
def calculate_influence(self, state):
    influence = np.zeros((self.rows, self.cols))
    active_mask = (state > 0) & (state <= self.n_a)

    for i in range(self.rows):
        for j in range(self.cols):
            if not active_mask[i, j]:
                continue

            dist_weight = np.exp(-self.distances[i, j] / self.R)
            dir_weight = 1 + self.w0 * self.directions[i, j]

            mask = self.distances[i, j] <= self.R
            influence += mask * dist_weight * dir_weight * (self.n_a - state[i, j] + 1) / self.n_a

    return influence
```

Figura 3 Cálculo de la influencia entre espectadores.

El cálculo de influencia considera la distancia entre espectadores usando una función exponencial. Se aplica un peso direccional para favorecer la propagación en sentido horario. Se utiliza máscaras para optimizar los cálculos.

3.2.2 Actualización los estados del sistema

La actualización de estados determina si un espectador debe activarse basado en su umbral. Se gestiona la transición entre estados activos y refractarios. Se maneja el retorno al estado de reposo.

La actualización del sistema se realiza en el método `update()`, que implementa las reglas de transición entre estados:

```
def update(self):
    influence = self.calculate_influence(self.states)
    new_states = np.copy(self.states)

    for i in range(self.rows):
        for j in range(self.cols):
            if self.states[i, j] == 0:
                if influence[i, j] > self.thresholds[i, j]:
                    new_states[i, j] = 1
            elif self.states[i, j] < self.n_a + self.n_r:
                new_states[i, j] = self.states[i, j] + 1
            else:
                new_states[i, j] = 0

    self.states = new_states
    self.frame_count += 1
    return self.states
```

Figura 4 Actualización de estados

El cálculo de influencia considera la distancia entre espectadores usando una función exponencial. Se aplica un peso direccional para favorecer la propagación en sentido horario. Se utiliza máscaras para optimizar los cálculos.

La actualización de estados determina si un espectador debe activarse basado en su umbral. Se gestiona la transición entre estados activos y refractarios. Se maneja el retorno al estado de reposo.

La actualización del sistema se realiza en el método `update()`, que implementa las reglas de transición entre estados:

- Estados activos.
- Estados refractarios.
- Estado de reposo.

3.2.3 Visualización de ola mexicana

La visualización se actualiza en cada iteración mediante la clase WaveSimulationGUI, que utiliza matplotlib para crear un mapa de calor dinámico. Este mapa representa:

- Estados activos con colores intensos.
- Estados refractarios con tonos intermedios.
- Estado de reposo con colores claros.

La simulación permite seguir la evolución de la ola en tiempo real, mostrando cómo una perturbación inicial se propaga a través del estadio, manteniendo una forma y velocidad estables que coinciden con las observaciones empíricas descritas en el artículo original.

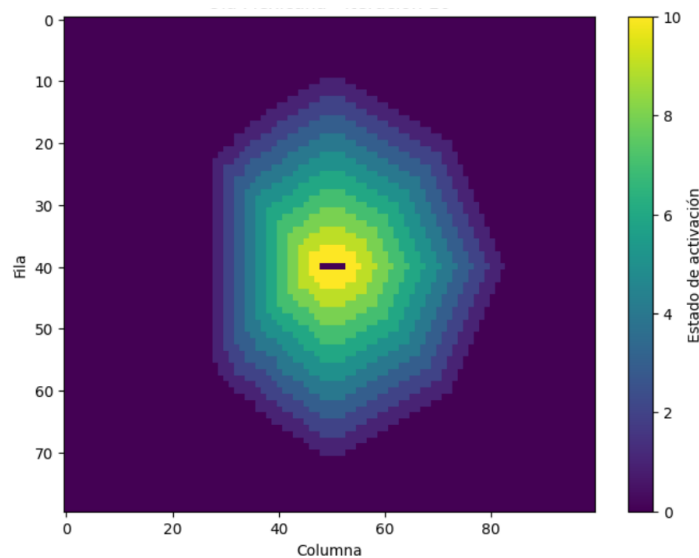


Figura 5 Visualización de ola mexicana.

3.3 Desafíos y soluciones

Entre los desafíos que tuvimos que resolver son los cálculos en las matrices de influencia, la visualización en tiempo real y el manejo de Estados. Se logró solucionar usando de máscaras y operaciones de NumPy, usando funciones como `clear_output()` y usando un sistema de estados numerados.

3.4 Resultados obtenidos

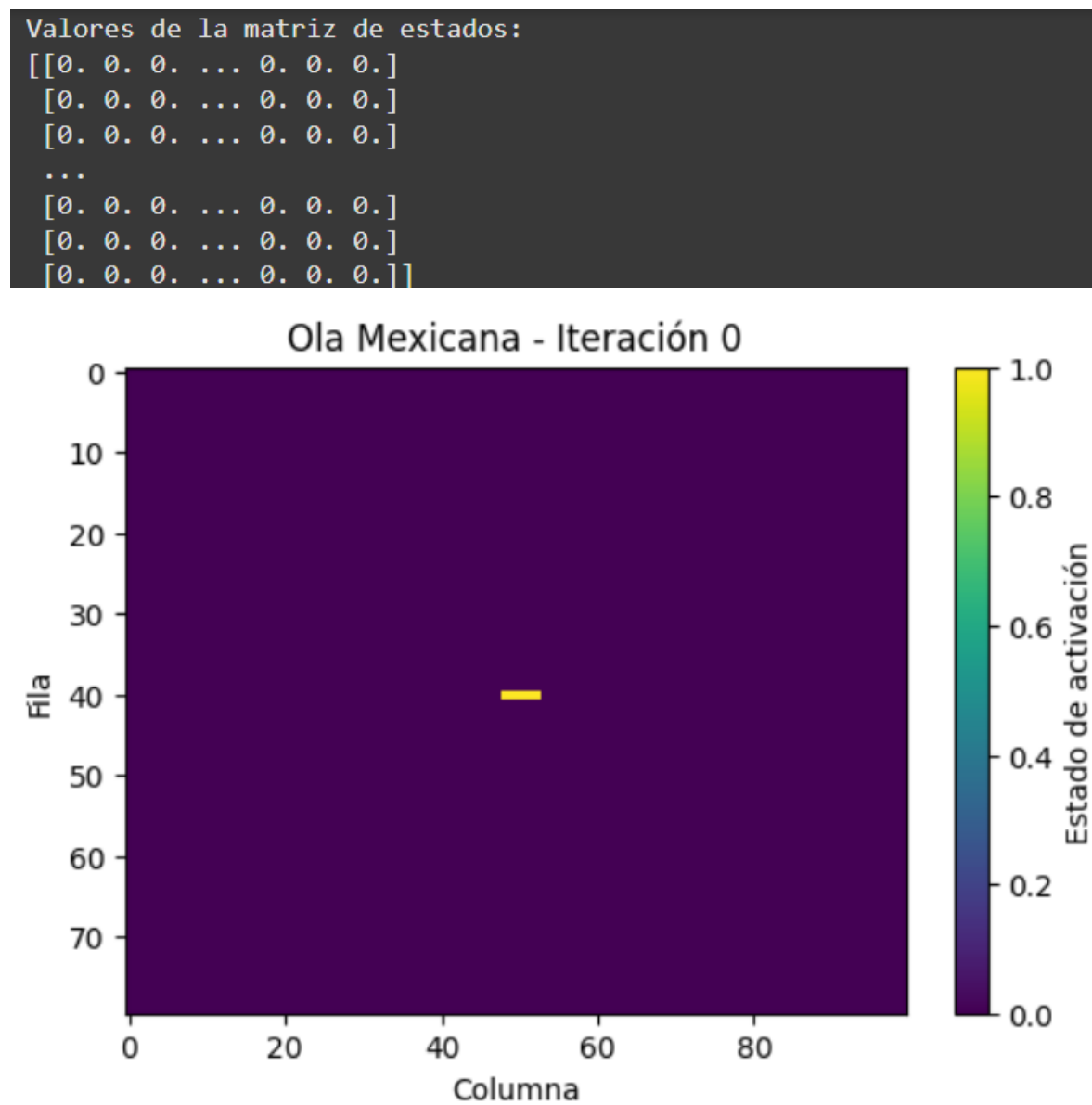


Figura 6 Visualización de Ola Mexicana – Iteración 0.

Valores de la matriz de estados:

```
[[0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]  
 ...  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]
```

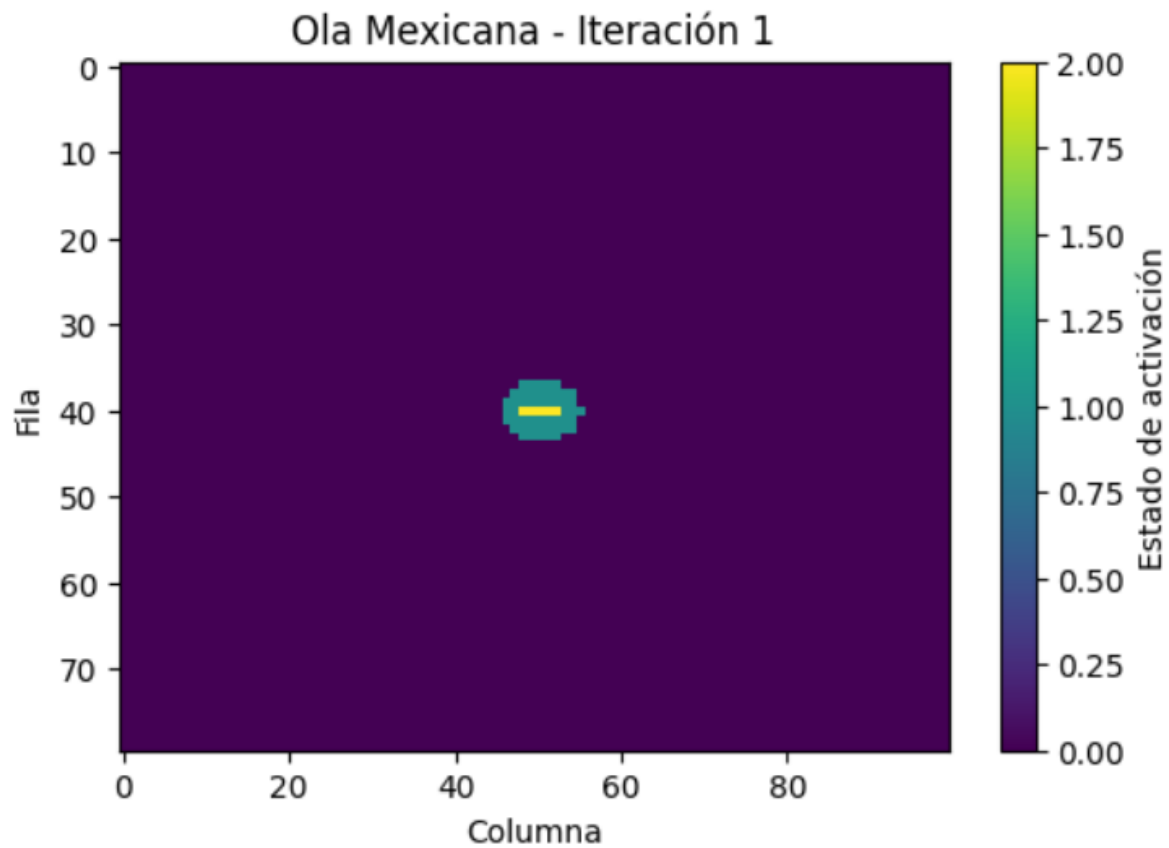


Figura 7 Visualización de Ola Mexicana – Iteración 1.

Frame: 10

Valores de la matriz de estados:

```
[[0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]  
 ...  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]]
```

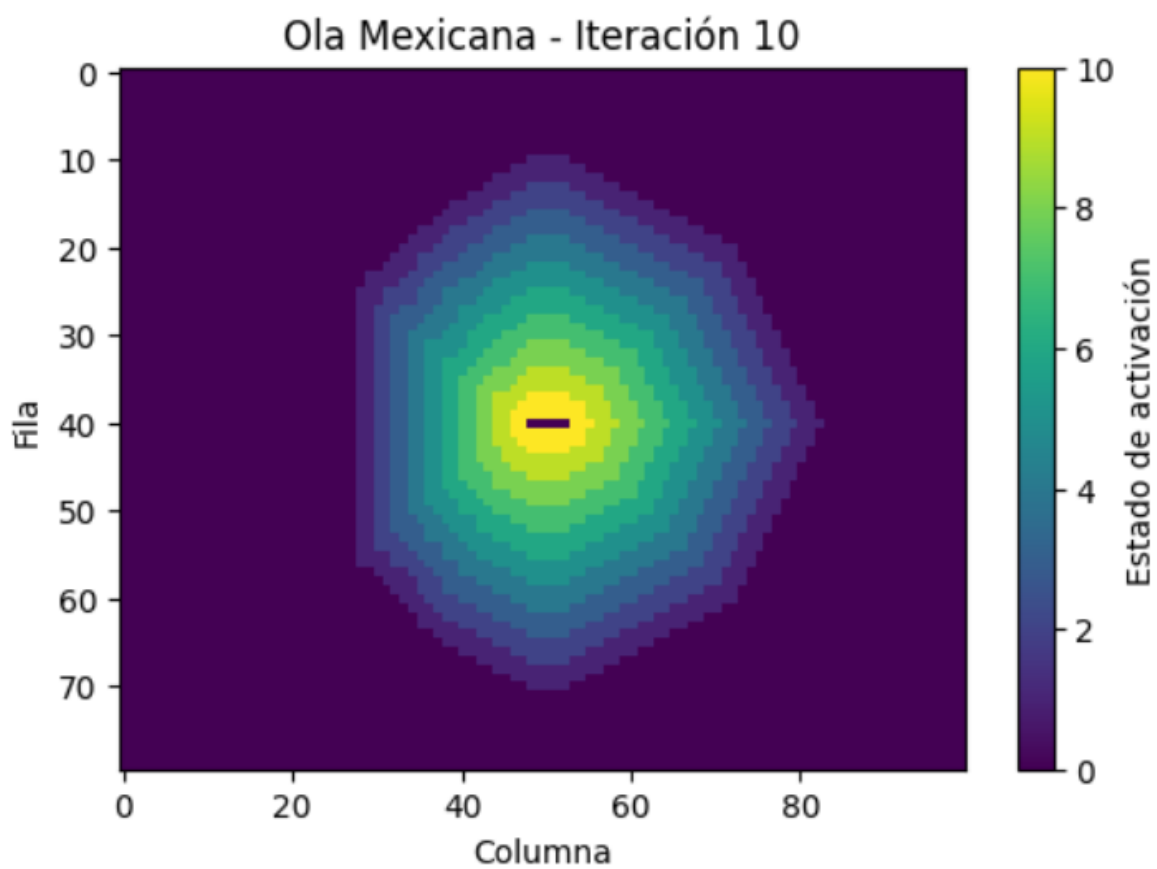


Figura 8 Visualización de Ola Mexicana – Iteración 10.

Frame: 22

Valores de la matriz de estados:

```
[[0. 0. 0. ... 1. 1. 0.]  
 [0. 0. 0. ... 1. 1. 0.]  
 [0. 0. 0. ... 1. 1. 1.]  
 ...  
 [0. 0. 0. ... 1. 1. 1.]  
 [0. 0. 0. ... 1. 1. 1.]  
 [0. 0. 0. ... 1. 1. 0.]]
```

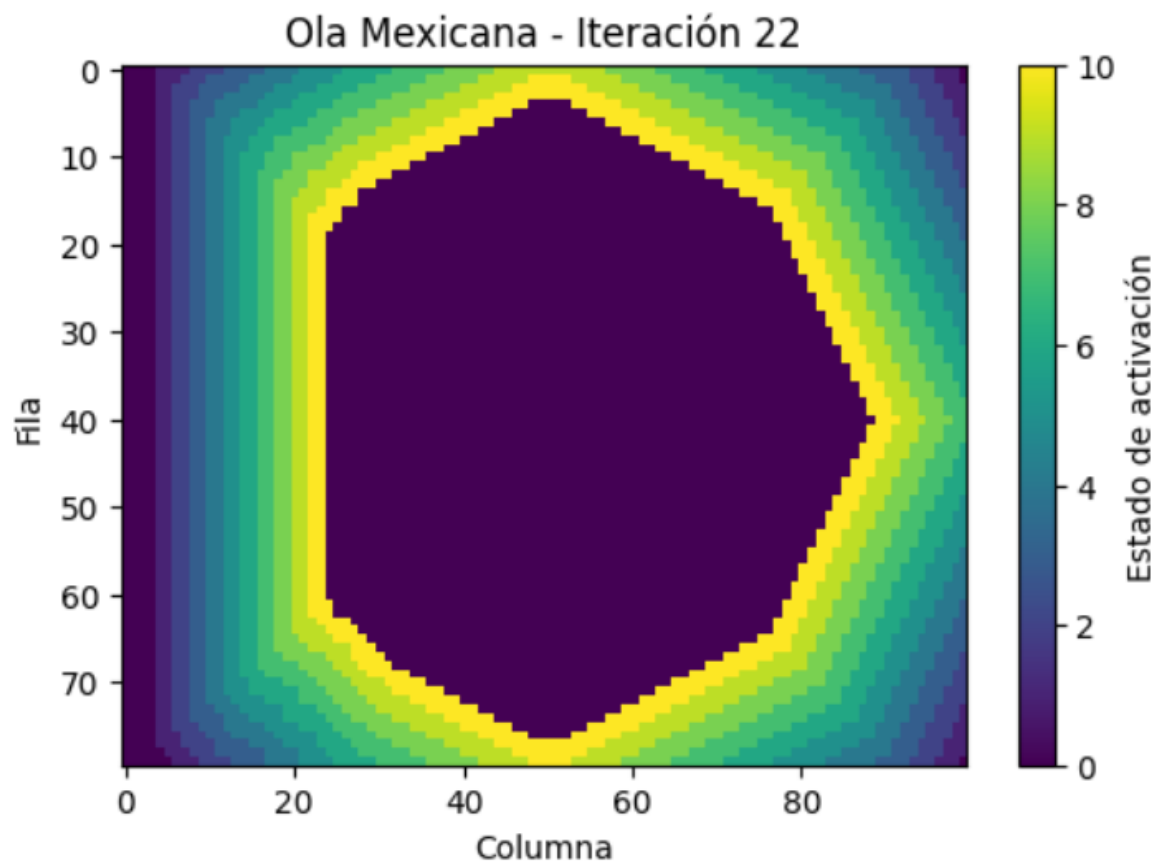


Figura 9 Visualización de Ola Mexicana – Iteración 22.

Valores de la matriz de estados:

```
[[7. 7. 8. ... 9. 9. 8.]  
 [7. 7. 8. ... 9. 9. 8.]  
 [7. 7. 8. ... 9. 9. 9.]  
 ...  
 [7. 7. 8. ... 9. 9. 9.]  
 [7. 7. 8. ... 9. 9. 9.]  
 [7. 7. 7. ... 9. 9. 8.]]
```

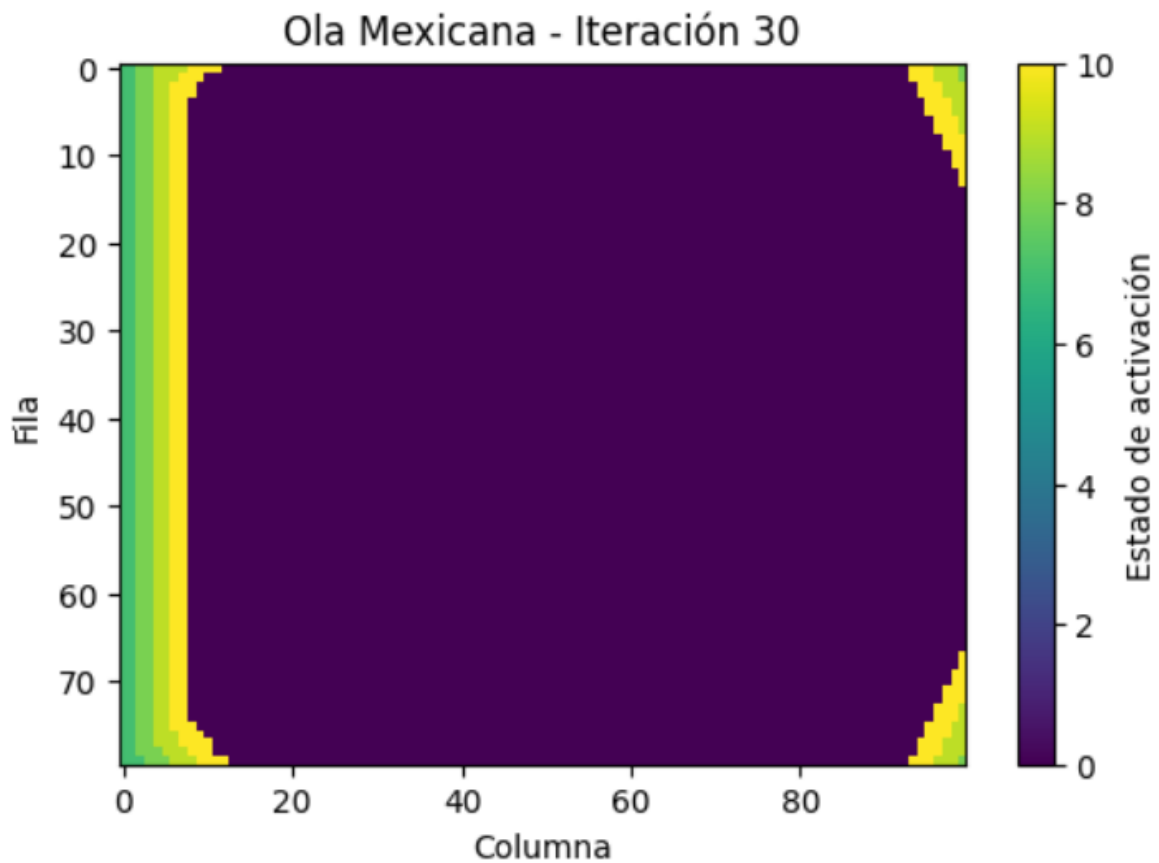


Figura 10 Visualización de Ola Mexicana – Iteración 30.

4 Conclusiones y recomendaciones.

La simulación de la Ola Mexicana es un ejemplo de cómo los modelos matemáticos y computacionales pueden ayudarnos a entender y reproducir comportamientos colectivos complejos. A través de esta práctica, se logró un modelo que captura las características de esta Ola Mexicana.

Durante la experimentación con diferentes parámetros, se observó que el comportamiento de la ola es sensible a los valores del umbral de activación (c) y el peso direccional (w_0). Se encontró que valores de umbral demasiado bajos pueden resultar en la propagación caótica de múltiples olas, mientras que valores demasiado altos pueden impedir la propagación por completo.

Para mejorar la práctica, se recomienda descubrir que pasa cuando se modifican los parámetros de influencia direccional. Por ejemplo, al reducir significativamente el peso direccional (w_0), se observaron ocasionalmente olas que se propagaban en direcciones alternativas o incluso olas que se dividían en múltiples frentes. Estos comportamientos emergentes sugieren que el modelo podría ser útil para estudiar otros tipos de comportamientos colectivos más allá de la ola tradicional.

El aprendizaje más significativo de esta práctica fue la comprensión de cómo comportamientos colectivos complejos pueden emerger de reglas relativamente simples a nivel individual. La simulación demuestra cómo la interacción local entre espectadores puede dar lugar a un patrón global coherente y auto-sostenible, proporcionando insights valiosos sobre la dinámica de sistemas sociales y la emergencia de comportamientos colectivos coordinados.

5 Bibliografía.

- Wiener, N. & Rosenblueth, A. Arch. Inst. Cardiol. Mexico 16, 205–265 (1946).
- Greenberg, J. M. & Hastings, S. P. SIAM J. Appl. Math. 34, 515–523 (1978).
- Bub, G., Shrier, A. & Glass, L. Phys. Rev. Lett. 88, 058101 (2002).